МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования «Московский технический университет связи и информатики»
Кафедра Программная инженерия

Отчет по лабораторной работе №2

“ Основы выборки SQL”

по дисциплине «Введение в информационные технологии»

Выполнил: студент группы
БПИ2501
Карпеко Никита Александрович
Проверила: Мосева Марина Сергеевна

Москва, 2026

Цель работы

Продолжить знакомство с основными возможностями SQL и научиться выполнять возможные задачи, связанные с выборкой данных.

Содержание для выполнения

Для выполнения работы воспользуйтесь схемой БД из лабораторной работы №5

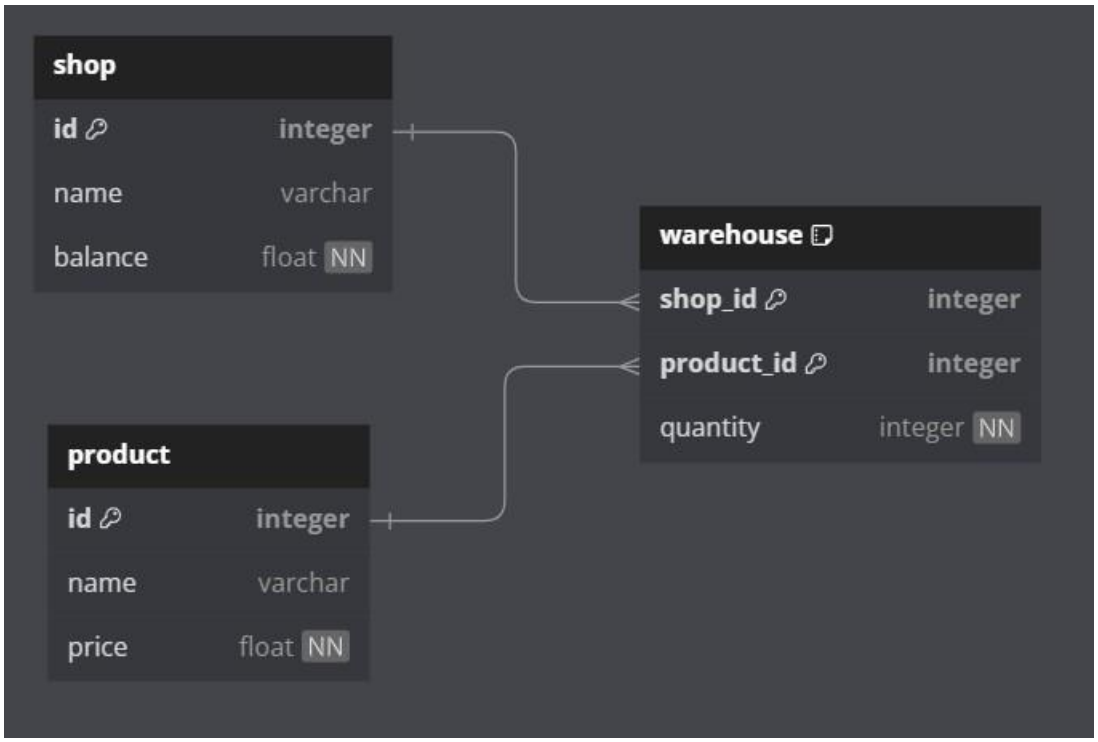


Рисунок 1 – Схемы базы данных Сортировка, оператор

Order By

При выполнении SELECT запроса, строки по умолчанию возвращаются в неопределённом порядке. Фактический порядок строк в этом случае зависит от плана соединения и сканирования, а также от порядка расположения данных на диске, поэтому полагаться на него нельзя. Для упорядочивания записей используется конструкция ORDER BY

```
SELECT поля_таблиц FROM наименование_таблицы
WHERE ...
ORDER BY столбец_1 [ASC | DESC][, столбец_n [ASC | DESC]]
```

Где ASC и DESC - направление сортировки:

- ASC - сортировка по возрастанию (по умолчанию)

- DESC - сортировка по убыванию

Для сортировки результатов по двум или более столбцам их следует указывать через запятую.

```
...ORDER BY столбец_1 [ASC | DESC], столбец_2 [ASC | DESC];
```

Группировка, оператор GROUP BY

Оператор GROUP BY в SQL используется для группировки строк из таблицы в более крупные наборы результатов на основании одного или нескольких столбцов. Эти группы включают разные записи в таблице и, соответственно, обладают разными характеристиками, которые нам могут быть весьма полезны.

Общая структура запроса с GROUP BY выглядит следующим образом:

```
SELECT [литералы, агрегатные_функции, поля_группировки]
FROM имя_таблицы
GROUP BY поля_группировки;
```

При использовании оператора GROUP BY мы перешли от работы с отдельными записями на работу с образовавшимися группами. В связи с этим мы не можем просто вывести любое поле из записи, как мы это могли делать раньше. Так как в каждой группе может быть несколько записей и в каждой из них в этом поле может быть разное значение.

При использовании GROUP BY мы можем выводить только:

- литералы, т.е. указанное явным образом фиксированные значения
- результаты агрегатных функций, т.е. вычисленные значения на основании набора значений.

Агрегатные функции

Агрегатная функция – это функция, которая выполняет вычисление на наборе значений и возвращает одиночное значение

Виды агрегатных функций

SUM(поле_таблицы)	Возвращает сумму значений
AVG(поле_таблицы)	Возвращает среднее значение
COUNT(поле_таблицы)	Возвращает количество записей
MIN(поле_таблицы)	Возвращает минимальное значение
MAX(поле_таблицы)	Возвращает максимальное значение

Ниже представлены несколько примеров использования агрегатных функций:

Найдём количество видов магазинов и отсортируем полученный список по убыванию:

```
SELECT name, COUNT(*) as amount FROM Shop GROUP BY
 name
ORDER BY amount DESC
```

Найдем средний остаток на балансе магазинов отсортированный по возрастанию

```
SELECT name, avg(balance) as avg_balance from shop group by name
order by avg_balance
```

Оператор HAVING

При использовании оператора GROUP BY в результате запроса формируются группы строк, а не отдельные записи таблицы. В связи с этим стандартный оператор фильтрации WHERE не может применяться для отбора значений, полученных с помощью агрегатных функций. Для фильтрации результатов агрегирования используется оператор HAVING. Он применяется после группировки данных и позволяет накладывать условия на уже сформированные группы.

Общая структура запроса с использованием HAVING:

```
SELECT поля, агрегатные_функции
FROM таблица, GROUP BY
поля, HAVING условие;
```

Например, следующий запрос выводит только те магазины, суммарный остаток товаров в которых превышает 50 единиц:

```
SELECT shop_id, SUM(quantity) AS total_quantity FROM warehouse
GROUP BY shop_id
HAVING SUM(quantity) > 50;
```

Оператор JOIN

В реляционных базах данных информация, относящаяся к одной предметной области, как правило, хранится в нескольких связанных таблицах. Для объединения данных из таких таблиц используется оператор JOIN. Оператор JOIN позволяет сформировать единый результат запроса на основе значений связанных столбцов, обычно первичного и внешнего ключей. Наиболее часто используется INNER JOIN, который возвращает только те строки, для которых существует соответствие в обеих таблицах:

```
SELECT shop.name, warehouse.quantity FROM warehouse
JOIN shop ON warehouse.shop_id = shop.id;
```

В данном примере к данным о количестве товаров на складе добавляется информация о названии магазина.

Также существует LEFT JOIN, который возвращает все строки из левой таблицы, даже если для них отсутствуют соответствующие записи в правой таблице. В этом случае отсутствующие значения заполняются NULL. Оператор JOIN широко применяется совместно с GROUP BY и агрегатными функциями для выполнения аналитических запросов, позволяющих получать обобщённую информацию на основе данных из нескольких таблиц.

Задания

1. Создать дополнительную таблицу “Сотрудник” выполнив следующую команду

```
CREATE TABLE worker (worker_id INTEGER PRIMARY KEY,
shop_id INTEGER REFERENCES shop(id), name VARCHAR(255),
salary INTEGER NOT NULL,
position VARCHAR(255));
```

2. Внести в таблицу данные.

3. К лабораторной работе прилагается файл data.sql. Импортировать данные в среде Jade, нажав на кнопку с тремя вертикальными точками в левом верхнем углу и выбрав пункт Import SQL.

4. К таблицам написать запросы операциями группировки и сортировки. Также реализовать запросы с использованием агрегатных функций для таблицы “Сотрудник”

5. Сформировать запрос, который выводит список сотрудников с указанием названия магазина, должности и зарплаты. Результат отсортировать по названию магазина, затем по зарплате по убыванию.

6. Составить запрос, который выводит для каждого магазина: название магазина, количество сотрудников, среднюю зарплату. Результат отсортировать по средней зарплате по убыванию.

7. Найти магазины, где средняя зарплата сотрудников выше 50 000. Вывести название магазина и среднюю зарплату.

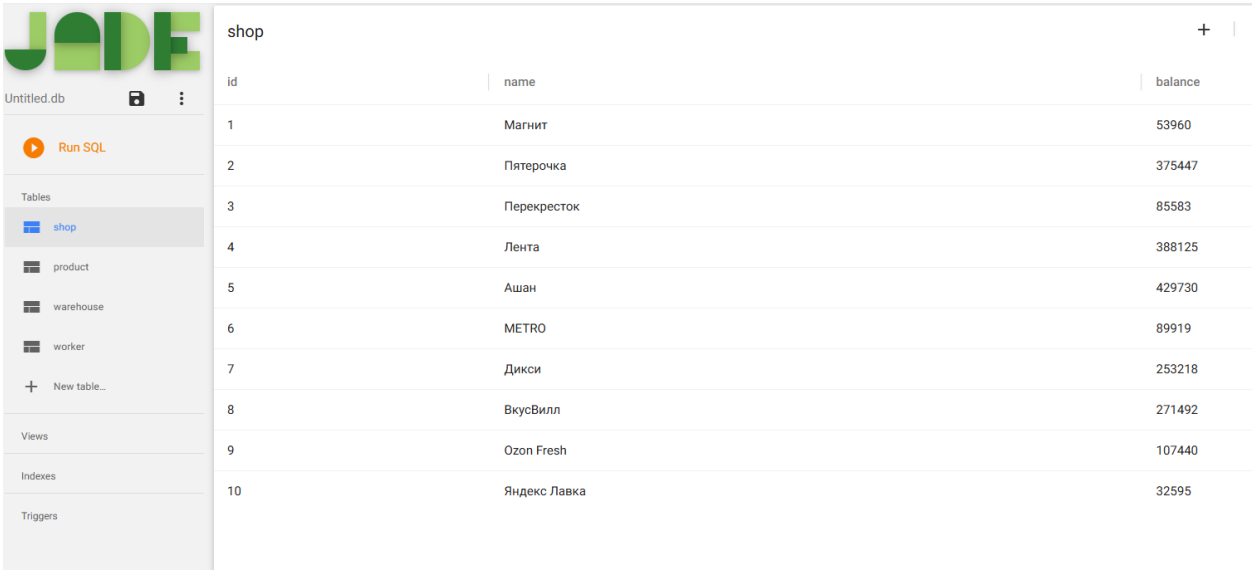
Общий код

```
CREATE TABLE shop (
id INTEGER PRIMARY KEY,
name VARCHAR(255),
balance FLOAT NOT NULL);

CREATE TABLE product (
id INTEGER PRIMARY KEY,
name VARCHAR(255),
price FLOAT NOT NULL);

CREATE TABLE warehouse (
shop_id INTEGER REFERENCES shop (id),
product_id INTEGER REFERENCES product (id),
quantity INTEGER NOT NULL);

CREATE TABLE worker (
worker_id INTEGER PRIMARY KEY,
shop_id INTEGER REFERENCES shop (id),
name VARCHAR(255),
salary INTEGER NOT NULL,
position VARCHAR(255));

SELECT * FROM shop
ORDER BY balance

SELECT * FROM product
ORDER BY name, price
```

```
SELECT shop_id, COUNT(product_id), quantity FROM warehouse
GROUP BY shop_id

SELECT shop_id, SUM(salary) as budjet, AVG(salary) FROM worker
GROUP BY shop_id
ORDER BY shop_id, budjet


SELECT worker.name, shop.name, position, salary FROM worker
JOIN shop ON worker.shop_id = shop.id
ORDER BY shop.name, salary DESC

SELECT shop.name, COUNT(worker.worker_id), AVG(worker.salary) as amount
FROM shop
JOIN worker ON shop.id = worker.shop_id
GROUP BY shop.name
ORDER BY amount DESC

SELECT shop.name, AVG(worker.salary) as amount FROM shop
JOIN worker ON shop.id = worker.shop_id
GROUP BY shop.name
HAVING amount > 50000
```

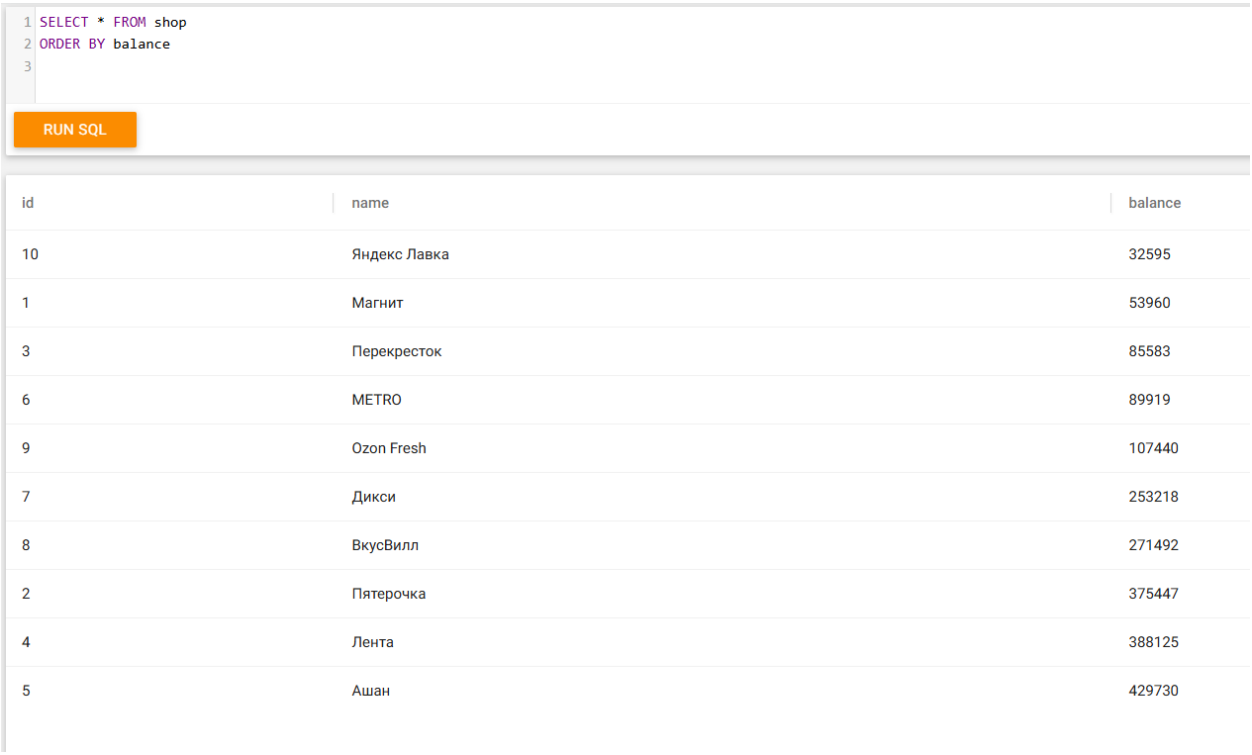
Ход выполнения

После создания таблиц БД и внесение данных (шаги 1-3)



id	name	balance
1	Магнит	53960
2	Пятерочка	375447
3	Перекресток	85583
4	Лента	388125
5	Ашан	429730
6	METRO	89919
7	Дикси	253218
8	ВкусВилл	271492
9	Ozon Fresh	107440
10	Яндекс Лавка	32595

Запросы операциями группировки и сортировки.



id	name	balance
10	Яндекс Лавка	32595
1	Магнит	53960
3	Перекресток	85583
6	METRO	89919
9	Ozon Fresh	107440
7	Дикси	253218
8	ВкусВилл	271492
2	Пятерочка	375447
4	Лента	388125
5	Ашан	429730

```sql
SELECT * FROM product
ORDER BY name, price
```

RUN SQL

id	name	price
24	Апельсины	3899
23	Бананы	386
45	Вино красное	2257
42	Вода минеральная	3965
43	Газированный напиток	4488
12	Говядина мякоть	654
30	Гречневая крупа	2564
49	Зубная паста	3858
8	Йогурт клубничный	3664
17	Картофель	455
35	Кетчуп	54
4	Кефир 2.5%	3754

```sql
SELECT shop_id, COUNT(product_id), quantity FROM warehouse
GROUP BY shop_id
```

RUN SQL

shop_id	COUNT(product_id)	quantity
1	16	119
2	13	87
3	13	199
4	6	135
5	10	155
6	13	98
7	8	60
8	6	56
9	9	161
10	6	3

```sql
SELECT shop_id, SUM(salary) as budjet, AVG(salary) FROM worker
GROUP BY shop_id
ORDER BY shop_id, budjet
```

RUN SQL

shop_id	budjet	AVG(salary)
1	820960	91217.77777777778
2	864795	72066.25
3	619647	77455.875
4	1291705	80731.5625
5	663175	82896.875
6	471531	67361.57142857143
7	927212	71324
8	729035	81003.88888888889
9	696141	77349
10	703200	78133.33333333333

Запрос, который выводит список сотрудников с указанием названия магазина, должности и зарплаты, отсортированный по названию магазина, затем по зарплате по убыванию. (шаг 5)

```sql
SELECT worker.name, shop.name, position, salary FROM worker
JOIN shop ON worker.shop_id = shop.id
ORDER BY shop.name, salary DESC
```

RUN SQL

name	name	position	salary
Сотрудник_14	METRO	Резчик-упаковщик	109753
Сотрудник_30	METRO	Менеджер	89409
Сотрудник_77	METRO	Охранник	62933
Сотрудник_95	METRO	Продавец-консультант	62897
Сотрудник_33	METRO	Товаровед	57579
Сотрудник_59	METRO	Мерчендайзер	45010
Сотрудник_35	METRO	Логист	43950
Сотрудник_92	Ozon Fresh	Менеджер	116239
Сотрудник_48	Ozon Fresh	Уборщик	94859
Сотрудник_83	Ozon Fresh	Специалист по приему товара	94032
Сотрудник_94	Ozon Fresh	Менеджер по аренде	87732
Сотрудник_75	Ozon Fresh	Менеджер по продажам	83705

Запрос, который выводит для каждого магазина: название магазина, количество сотрудников, среднюю зарплату. Результат отсортировать по средней зарплате по убыванию. (шаг 6)

```sql
SELECT shop.name, COUNT(worker.worker_id), AVG(worker.salary) as amount FROM shop
JOIN worker ON shop.id = worker.shop_id
GROUP BY shop.name
ORDER BY amount DESC
```

RUN SQL

name	COUNT(worker.worker_id)	amount
Магнит	9	91217.77777777778
Ашан	8	82896.875
ВкусВилл	9	81003.88888888889
Лента	16	80731.5625
Яндекс Лавка	9	78133.33333333333
Перекресток	8	77455.875
Ozon Fresh	9	77349
Пятерочка	12	72066.25
Дикси	13	71324
METRO	7	67361.57142857143

Запрос на магазины, где средняя зарплата сотрудников выше 50 000. Вывести название магазина и среднюю зарплату. (шаг 7)

```
1 SELECT shop.name, AVG(worker.salary) as amount FROM shop
2 JOIN worker ON shop.id = worker.shop_id
3 GROUP BY shop.name
4 HAVING amount > 50000
```

RUN SQL

name	amount
METRO	67361.57142857143
Ozon Fresh	77349
Ашан	82896.875
ВкусВилл	81003.88888888889
Дикси	71324
Лента	80731.5625
Магнит	91217.77777777778
Перекресток	77455.875
Пятерочка	72066.25
Яндекс Лавка	78133.33333333333

Заключение

Снова получилось провзаимодействовать с возможностями SQL и попробовать выполнить возможные задачи, связанные с сортировкой и группировкой данных.